Codul analizei MQTTset

Scripturile care reproduc toate cifrele și graficele din document. Repo: https://github.com/Darius15222/mqtt-attack-analysis (folderul cod/); site: https://darius15222.github.io/mqtt-attack-analysis/

Rulare: `python3 -m venv venv && venv/bin/pip install pandas scipy statsmodels mlxtend matplotlib`, apoi `venv/bin/python 01_cap1-7_statistici_corelatii.py mqttdataset_reduced.csv.zip`. Arborii: `Rscript 09_cap10_arbori_ctree.R mqttdataset_reduced.csv.zip arbori` (pachetul R partykit).

1. `01_cap1-7_statistici_corelatii.py`
2. `02_cap7-9_regresii_logistica.py`
3. `03_cap9_probabilitati.py`
4. `04_cap9_firth.py`
5. `05_cap8-9_cook_durbin_watson.py`
6. `06_cap11_reguli_asociere.py`
7. `07_grafice_corelatie.py`
8. `08_grafice_restul.py`
9. `09_cap10_arbori_ctree.R`
10. `10_grafice_arbori.py`

01_cap1-7_statistici_corelatii.py

```python
"""Statistici descriptive, teste și corelații pentru MQTTset (cap. 1-7 din raport).

Reproduce cifrele din:
  - distribuția claselor și coloanele constante;
  - statisticile descriptive (cap. 5) și valorile extreme după regula IQR;
  - tabelul pe clase, testul Kruskal-Wallis, χ² pe tcp.flags și mqtt.conack.val,
    testul Mann-Whitney flood vs. legitimate (cap. 6);
  - corelațiile Spearman/Pearson și verificările din cap. 7;
  - exemplele de rânduri și tabelul cu valori tipice normal/atac (cap. 4).

Intrare:  fișierul CSV MQTTset (merge și .zip, pandas îl citește direct).
Ieșire:   doar text la stdout.

Rulare:
    python 01_cap1-7_statistici_corelatii.py ../mqttdataset_reduced.csv.zip
"""
import sys

import pandas as pd
from scipy import stats

date = pd.read_csv(sys.argv[1], low_memory=False)

# Cele 13 coloane numerice folosite în matricea de corelație (cap. 7).
COLOANE_NUMERICE = [
    "tcp.time_delta", "tcp.len", "mqtt.len", "mqtt.kalive", "mqtt.qos",
    "mqtt.dupflag", "mqtt.retain", "mqtt.conflag.cleansess", "mqtt.conflag.passwd",
    "mqtt.conflag.uname", "mqtt.proto_len", "mqtt.ver", "mqtt.conack.val",
]

# Indicatorii binari / aproape constanți: îi descriem prin procentul de valori nenule.
INDICATORI = [
    "mqtt.dupflag", "mqtt.retain", "mqtt.conflag.cleansess", "mqtt.conflag.passwd",
    "mqtt.conflag.uname", "mqtt.proto_len", "mqtt.ver", "mqtt.conack.val", "mqtt.qos",
]

# --- Cap. 1-3: dimensiunea setului, clasele, coloanele constante ---
print(date.shape)
print(date.target.value_counts(), (date.target.value_counts(normalize=True) * 100).round(2))

coloane_constante = [c for c in date if date[c].nunique() == 1]
print("const", len(coloane_constante), coloane_constante)

# --- Cap. 5: statistică descriptivă pentru coloanele continue ---
for coloana in ["tcp.len", "tcp.time_delta", "mqtt.len", "mqtt.kalive"]:
    x = date[coloana]
    print(coloana,
          "mean", round(x.mean(), 2),
          "std", round(x.std(), 2),
          "med", x.median(),
          "q", x.quantile([.25, .75]).tolist(),
          "skew", round(x.skew(), 2),
          "kurt", round(x.kurt(), 1))

    # Valori extreme după regula IQR: în afara [Q1 - 1,5·IQR, Q3 + 1,5·IQR].
    q1, q3 = x.quantile([.25, .75])
    iqr = q3 - q1
    prag_sus = q3 + 1.5 * iqr
    extreme = (x < q1 - 1.5 * iqr) | (x > prag_sus)
    print("  IQR: peste", prag_sus,
          "n", extreme.sum(),
          round(extreme.mean() * 100, 2), "%",
          date.target[extreme].value_counts(normalize=True).round(3).to_dict())

# Coloanele categoriale: valoarea cea mai frecventă și ponderea ei.
# Le citim ca text pentru că amestecă numere ("0") cu valori hex ("0x00000010") și NaN.
for coloana in ["tcp.flags", "mqtt.hdrflags", "mqtt.msgtype", "mqtt.conflags",
                "mqtt.protoname", "mqtt.conack.flags", "mqtt.msg"]:
    frecvente = date[coloana].astype(str).value_counts(normalize=True)
    print(coloana, frecvente.index[0], round(frecvente.iloc[0] * 100, 2))

```

```python
for coloana in INDICATORI:
    nenule = date[coloana] != 0
    print(coloana, "nonzero%", round(nenule.mean() * 100, 2), "count", nenule.sum(),
          date[coloana].unique()[:6])

# --- Cap. 6: comparații pe clase ---
legitim = date[date.target == "legitimate"]
tcp_len_legitim = legitim["tcp.len"]
print("leg tcp.len med q", tcp_len_legitim.median(), tcp_len_legitim.quantile([.25, .75]).tolist(),
      "mean", round(tcp_len_legitim.mean(), 2),
      "zero%", round((tcp_len_legitim == 0).mean() * 100, 1),
      "10-14%", round(tcp_len_legitim.between(10, 14).mean() * 100, 1))

# Mediana și quartilele pe clase, apoi Kruskal-Wallis (date puternic asimetrice, deci test neparametric).
for coloana in ["tcp.time_delta", "tcp.len", "mqtt.len"]:
    pe_clase = date.groupby("target")[coloana]
    print(coloana, {clasa: (g.median(), g.quantile(.25), g.quantile(.75)) for clasa, g in pe_clase})
    print(" KW H", stats.kruskal(*[g.values for _, g in pe_clase]))

for coloana in ["tcp.flags", "mqtt.hdrflags", "mqtt.msgtype"]:
    for clasa, grup in date.groupby("target"):
        frecvente = grup[coloana].astype(str).value_counts(normalize=True)
        print(" ", coloana, clasa, frecvente.index[0], round(frecvente.iloc[0] * 100, 2))

for coloana in ["mqtt.dupflag", "mqtt.retain", "mqtt.conflag.cleansess", "mqtt.conflag.passwd",
                "mqtt.conflag.uname", "mqtt.conack.val", "mqtt.qos"]:
    procent_nenule = date.groupby("target")[coloana].apply(lambda s: (s != 0).mean() * 100)
    print(coloana, procent_nenule.round(2).to_dict())

# 65535 = keep-alive maxim din MQTT, specific atacului slowite.
print("kalive slowite 65535%",
      round((date[date.target == "slowite"]["mqtt.kalive"] == 65535).mean() * 100, 2),
      date["mqtt.kalive"].unique()[:10])

# Testul χ² de independență: tcp.flags × clasă.
observate = pd.crosstab(date["tcp.flags"], date.target)
print(observate)
chi2_flags = stats.chi2_contingency(observate)
print("chi2", chi2_flags[0], chi2_flags[2])
asteptate = chi2_flags[3]
print("E FIN,ACK bf",
      asteptate[observate.index.get_loc("0x00000011"), observate.columns.get_loc("bruteforce")])

# Testul χ² pentru mqtt.conack.val × clasă (și frecvențele așteptate).
observate_conack = pd.crosstab(date["mqtt.conack.val"], date.target)
chi2_conack = stats.chi2_contingency(observate_conack)
print(observate_conack, "chi2", chi2_conack[0])
print("E", chi2_conack[3])

# Mann-Whitney pe tcp.time_delta: legitimate vs. flood, în ambele ordini (U se schimbă cu ordinea).
delta_legitim = legitim["tcp.time_delta"]
delta_flood = date[date.target == "flood"]["tcp.time_delta"]
print("MW", stats.mannwhitneyu(delta_legitim, delta_flood), len(delta_legitim) * len(delta_flood))
print("MW flipped", stats.mannwhitneyu(delta_flood, delta_legitim))

# --- Cap. 4: rânduri de exemplu (numerotate de la 1, ca în CSV) ---
for clasa in date.target.unique():
    print(clasa, list(date.index[date.target == clasa][:2] + 1))
print(date.loc[[0, 4, 1, 46, 36, 6, 1420], ["tcp.flags", "tcp.len", "mqtt.len", "target"]])

# --- Cap. 7: corelații Spearman / Pearson ---
spearman = date[COLOANE_NUMERICE].corr("spearman")
print(spearman.round(2))

date["is_attack"] = (date.target != "legitimate").astype(int)
perechi = [
    ("tcp.len", "mqtt.len"), ("tcp.time_delta", "tcp.len"), ("tcp.time_delta", "mqtt.len"),
    ("tcp.len", "is_attack"), ("mqtt.len", "is_attack"), ("tcp.time_delta", "is_attack"),
    ("mqtt.qos", "is_attack"), ("mqtt.dupflag", "is_attack"),
]
for a, b in perechi:
    print(a, b, round(date[a].corr(date[b]), 3), round(date[a].corr(date[b], method="spearman"), 3))

# Câmpurile de conectare (mqtt.ver, mqtt.kalive) sunt nenule exact pe aceleași rânduri.
```

```
ver_nenul = date["mqtt.ver"] != 0
print("conn nonzero rows", ver_nenul.sum(), (ver_nenul == (date["mqtt.kalive"] != 0)).all())

qos1 = date[date["mqtt.qos"] == 1]
print("qos1", len(qos1), qos1.target.unique(), qos1["mqtt.len"].min(), qos1["mqtt.len"].max(),
      date[date["mqtt.qos"] == 0]["mqtt.len"].mean())
# dupflag = 1 apare doar pe rânduri cu qos = 1 (rezultatul așteptat e 0).
print("dup1 subset qos1", ((date["mqtt.dupflag"] == 1) & (date["mqtt.qos"] != 1)).sum())

for clasa, grup in date.groupby("target"):
    constante_in_clasa = [c for c in COLOANE_NUMERICE if grup[c].nunique() == 1]
    print(clasa, "tcp-mqtt spearman",
          round(grup["tcp.len"].corr(grup["mqtt.len"], method="spearman"), 2),
          "nconst", len(constante_in_clasa), constante_in_clasa)

flood = date[date.target == "flood"]
print("tcp.len max flood", flood["tcp.len"].max(), "mqtt.len flood max", flood["mqtt.len"].max())

# Sumele din formula coeficientului Pearson, calculate de mână pentru exemplul din raport.
x = date["tcp.len"]
y = date["mqtt.len"]
print("sxy", ((x - x.mean()) * (y - y.mean())).sum(),
      "sxx", ((x - x.mean()) ** 2).sum(),
      "syy", ((y - y.mean()) ** 2).sum())

# --- Cap. 4: tabelul cu valori tipice „normal” vs. „atac” ---
# Pentru atac luăm cea mai frecventă valoare care apare (aproape) numai în atacuri:
# cel puțin 99% din rândurile cu acea valoare sunt atacuri.
atacuri = date[date.is_attack == 1]
for coloana in ["tcp.flags", "tcp.time_delta", "tcp.len", "mqtt.hdrflags", "mqtt.msgtype",
                "mqtt.len", "mqtt.qos", "mqtt.dupflag", "mqtt.kalive"]:
    valori_text = date[coloana].astype(str)
    pondere_atac = date.groupby(valori_text)["is_attack"].mean()
    valori_de_atac = pondere_atac[pondere_atac >= 0.99].index

    frecvente_atac = atacuri[coloana].astype(str).value_counts()
    frecvente_atac = frecvente_atac[frecvente_atac.index.isin(valori_de_atac)]
    frecvente_normal = legitim[coloana].astype(str).value_counts(normalize=True)

    if len(frecvente_atac):
        exemplu_atac = (frecvente_atac.index[0], round(frecvente_atac.iloc[0] / len(atacuri) * 100, 1))
    else:
        exemplu_atac = None
    print("T4", coloana,
          "normal", frecvente_normal.index[0], round(frecvente_normal.iloc[0] * 100, 1),
          "attack", exemplu_atac)
```

02_cap7-9_regresii_logistica.py

```python
"""Corelații pe clase, regresii liniare, VIF și regresie logistică pentru MQTTset (cap. 7-9).

Reproduce cifrele din:
  - matricea Spearman pe tot setul și pe fiecare clasă (cap. 7);
  - verificările pe flood / tcp.len > 1460 și valorile maxime (cap. 8);
  - regresiile liniare 1-3, inclusiv variantele logaritmice (cap. 8-9);
  - VIF pentru 5 și 3 predictori;
  - regresia logistică: modelul final cu 3 predictori, separarea modelului cu 5 predictori
    (pașii 10-30), probabilitățile prezise pe clase, tabelul 9.2, msgtype pe clase (cap. 9).

Intrare:  fișierul CSV MQTTset (merge și .zip, pandas îl citește direct).
Ieșire:   doar text la stdout.

Rulare:
    python 02_cap7-9_regresii_logistica.py ../mqttdataset_reduced.csv.zip
"""
import sys

import numpy as np
import pandas as pd
import statsmodels.api as sm
from scipy import stats
from statsmodels.stats.outliers_influence import variance_inflation_factor as vif

date = pd.read_csv(sys.argv[1], low_memory=False)
date["is_attack"] = (date.target != "legitimate").astype(int)

# Cele 13 coloane numerice folosite în matricea de corelație (cap. 7).
COLOANE_NUMERICE = [
    "tcp.time_delta", "tcp.len", "mqtt.len", "mqtt.kalive", "mqtt.qos",
    "mqtt.dupflag", "mqtt.retain", "mqtt.conflag.cleansess", "mqtt.conflag.passwd",
    "mqtt.conflag.uname", "mqtt.proto_len", "mqtt.ver", "mqtt.conack.val",
]
PREDICTORI_5 = ["tcp.len", "mqtt.len", "tcp.time_delta", "mqtt.qos", "mqtt.dupflag"]
PREDICTORI_3 = ["tcp.len", "tcp.time_delta", "mqtt.len"]

# --- Cap. 7: matricea Spearman ---
pd.set_option("display.width", 250)  # ca matricea 13×13 să încapă pe un rând
spearman = date[COLOANE_NUMERICE].corr("spearman")
print(spearman.round(2).to_string())

# Pe fiecare clasă: câte perechi de coloane sunt practic identice (ρ ≥ 0,995).
# Coloanele constante în clasă sunt scoase, altfel corelația lor e NaN.
for clasa, grup in date.groupby("target"):
    coloane_variabile = [c for c in COLOANE_NUMERICE if grup[c].nunique() > 1]
    matrice = grup[coloane_variabile].corr("spearman").values
    deasupra_diagonalei = np.triu_indices(len(coloane_variabile), 1)
    print(clasa,
          "pairs", len(deasupra_diagonalei[0]),
          "r>=0.995", (matrice[deasupra_diagonalei] >= 0.995).sum())

print("pearson qos-mqtt.len", date["mqtt.qos"].corr(date["mqtt.len"]))

# --- Cap. 8: verificări pe flood și pe valorile mari ---
# 1460 = payload-ul TCP maxim într-un cadru Ethernet (MSS tipic).
flood = date[date.target == "flood"]
print("flood mqtt.len when tcp.len>1460", flood[flood["tcp.len"] > 1460]["mqtt.len"].describe().to_dict())
print("flood ACK empty", ((flood["tcp.flags"] == "0x00000010") & (flood["tcp.len"] == 0)).sum(),
      "tcp.len>1460 all", (date["tcp.len"] > 1460).sum(),
      date[date["tcp.len"] > 1460].target.unique())
print("legit tcp.len max", date[date.target == "legitimate"]["tcp.len"].max(),
      "time_delta max", date["tcp.time_delta"].max(),
      (date["tcp.time_delta"] > 50).sum())

# --- Regresia 1: mqtt.len ~ tcp.len (simplă și pe log(1 + x)) ---
regresia1 = sm.OLS(date["mqtt.len"], sm.add_constant(date[["tcp.len"]])).fit()
print("R1", regresia1.params.round(4).to_dict(), round(regresia1.rsquared, 4),
      "ssr", regresia1.ssr,
      "res skew/kurt", round(stats.skew(regresia1.resid), 2), round(stats.kurtosis(regresia1.resid), 1))

date_log = np.log1p(date[["tcp.len", "mqtt.len"]])  # log1p pentru că există multe valori 0
```

```
regresia1_log = sm.OLS(date_log["mqtt.len"], sm.add_constant(date_log["tcp.len"])).fit()
print("R1log", regresia1_log.params.round(3).to_dict(), round(regresia1_log.rsquared, 3),
      round(stats.skew(regresia1_log.resid), 2), round(stats.kurtosis(regresia1_log.resid), 1))

# --- Regresia 2: tcp.time_delta ~ tcp.len + mqtt.len ---
regresia2 = sm.OLS(date["tcp.time_delta"], sm.add_constant(date[["tcp.len", "mqtt.len"]])).fit()
print("R2", regresia2.params.to_dict(), regresia2.pvalues.to_dict(), regresia2.rsquared,
      stats.skew(regresia2.resid), stats.kurtosis(regresia2.resid))

# --- Regresia 3: regresia 1 pe fiecare clasă (simplă și logaritmică) ---
for clasa, grup in date.groupby("target"):
    simpla = sm.OLS(grup["mqtt.len"], sm.add_constant(grup["tcp.len"])).fit()
    grup_log = np.log1p(grup[["tcp.len", "mqtt.len"]])
    logaritmica = sm.OLS(grup_log["mqtt.len"], sm.add_constant(grup_log["tcp.len"])).fit()
    print("R3", clasa, len(grup), simpla.params.round(4).tolist(),
          round(simpla.rsquared, 3), round(logaritmica.rsquared, 3))

# --- VIF (multicoliniaritate); indicele 0 e constanta, deci pornim de la 1 ---
X5 = sm.add_constant(date[PREDICTORI_5]).values
print("VIF5", [round(vif(X5, i), 2) for i in range(1, 6)])
X3 = sm.add_constant(date[PREDICTORI_3]).values
print("VIF3", [round(vif(X3, i), 2) for i in range(1, 4)])

# --- Cap. 9: regresia logistică is_attack ~ 3 predictori (modelul final) ---
logistica3 = sm.GLM(date.is_attack, sm.add_constant(date[PREDICTORI_3]),
                    family=sm.families.Binomial()).fit()
print("LOG6", logistica3.params.round(4).to_dict(), logistica3.bse.round(4).to_dict(),
      logistica3.tvalues.round(2).to_dict(),
      logistica3.null_deviance, logistica3.deviance, logistica3.aic,
      "iters", logistica3.fit_history["iteration"])

# Modelul cu 5 predictori are separare completă: coeficienții și erorile standard cresc
# la fiecare pas. tol=1e-30 împiedică oprirea timpurie, ca să vedem pașii 10-30.
for nr_pasi in [10, 15, 20, 25, 30]:
    logistica5 = sm.GLM(date.is_attack, sm.add_constant(date[PREDICTORI_5]),
                        family=sm.families.Binomial()).fit(maxiter=nr_pasi, tol=1e-30)
    print("LOG5 it", nr_pasi, logistica5.params.round(3).to_dict(), logistica5.bse.round(1).to_dict(),
          round(logistica5.deviance, 1))

# Probabilitățile prezise de modelul final, pe clase.
# „~0.42” = ponderea rândurilor cu probabilitatea ≈ 0,4235 (valoarea discutată în 9.6).
probabilitati = pd.Series(logistica3.predict(), index=date.index)
for clasa in date.target.unique():
    p = probabilitati[date.target == clasa]
    print("prob", clasa,
          "max", round(p.max(), 3),
          "med", round(p.median(), 3),
          ">0.9", round((p > 0.9).mean(), 3),
          "~0.42", round((np.abs(p - 0.4235) < 0.005).mean(), 3))

# --- Tabelul 9.2: mediana și quartilele pentru atacuri ---
atacuri = date[date.is_attack == 1]
for coloana in ["tcp.time_delta", "tcp.len", "mqtt.len"]:
    print("9.2", coloana, atacuri[coloana].median(), atacuri[coloana].quantile([.25, .75]).tolist())

print("msgtype by class", pd.crosstab(date["mqtt.msgtype"], date.target).to_string())
```

03_cap9_probabilitati.py

```python
"""Probabilitățile prezise de regresia logistică și verificările din cap. 9 pentru MQTTset.

Reproduce cifrele din:
  - 9.6: probabilitățile prezise pe clase și curva logistică pentru un pachet tipic;
  - tabelul 9.2: mediana și quartilele pentru atacuri;
  - mqtt.msgtype pe clase și pachetele PUBLISH+ACK din dos;
  - câteva calcule de verificare citate în text (tcp.time_delta = 3e-6,
    exemplul de asimetrie/aplatizare, contribuția χ² pentru conack).

Intrare:  fișierul CSV MQTTset (merge și .zip, pandas îl citește direct).
Ieșire:   doar text la stdout.

Rulare:
    python 03_cap9_probabilitati.py ../mqttdataset_reduced.csv.zip
"""
import sys

import numpy as np
import pandas as pd
import statsmodels.api as sm

date = pd.read_csv(sys.argv[1], low_memory=False)
date["is_attack"] = (date.target != "legitimate").astype(int)

# --- 9.6: modelul logistic final și probabilitățile prezise pe clase ---
model = sm.GLM(date.is_attack, sm.add_constant(date[["tcp.len", "tcp.time_delta", "mqtt.len"]]),
               family=sm.families.Binomial()).fit()
probabilitati = pd.Series(np.asarray(model.predict()), index=date.index)
for clasa in date.target.unique():
    p = probabilitati[date.target == clasa]
    print("prob", clasa,
          "max", round(p.max(), 3),
          "med", round(p.median(), 3),
          ">0.9", round((p > 0.9).mean(), 3),
          "~0.42", round((np.abs(p - 0.4235) < 0.005).mean(), 3))

# Probabilitatea pentru un pachet tipic: tcp.len = 13, mqtt.len = 11, tcp.time_delta = 0.
coef = model.params
print("curve d=0 typical", 1 / (1 + np.exp(-(coef.const + coef["tcp.len"] * 13 + coef["mqtt.len"] * 11))))

# --- Tabelul 9.2: mediana și quartilele pentru atacuri ---
atacuri = date[date.is_attack == 1]
for coloana in ["tcp.time_delta", "tcp.len", "mqtt.len"]:
    print("9.2", coloana, atacuri[coloana].median(), atacuri[coloana].quantile([.25, .75]).tolist())

# --- mqtt.msgtype pe clase ---
print(pd.crosstab(date["mqtt.msgtype"], date.target).to_string())

# Pachetele dos de tip PUBLISH (msgtype 3) cu flag-ul TCP ACK (0x10).
dos_publish_ack = date[(date.target == "dos")
                       & (date["mqtt.msgtype"] == 3)
                       & (date["tcp.flags"] == "0x00000010")]
print("dos publish ACK", len(dos_publish_ack), dos_publish_ack["tcp.len"].unique()[:5],
      "70%", round(len(dos_publish_ack) * .7))

# --- Verificări pentru cifre citate în text ---
print("time_delta =3e-6 share", (date["tcp.time_delta"] == 3e-6).mean(),
      "<=3e-6", (date["tcp.time_delta"] <= 3e-6).mean())

# Exemplul mic de asimetrie și aplatizare din raport.
exemplu = pd.Series([1, 2, 2, 3, 10])
print(exemplu.skew(), exemplu.kurt())

# Contribuția unei celule la χ²: (O - E)² / E, cu O = 1428 și E = 62,5742.
print("conack contrib", (1428 - 62.5742) ** 2 / 62.5742)
```

04_cap9_firth.py

```python
"""
Regresia logistică Firth (cap. 9.5 din tema MQTTset).

Reproduce: coeficienții, erorile standard și statisticile Chisq
(testul raportului de verosimilitate penalizată) pentru fiecare predictor.

Intrare:  CSV-ul MQTTset (sau arhiva .zip, pandas o citește direct).
Ieșire:   doar text la consolă.

Rulare:
    python 04_cap9_firth.py ../mqttdataset_reduced.csv.zip

Firth este implementat manual (numpy), fără pachete suplimentare.
Motivul folosirii Firth: datele au separare (aproape) completă, iar regresia
logistică obișnuită dă coeficienți care fug la infinit. Penalizarea Firth
ține coeficienții finiți.
"""
import sys

import numpy as np
import pandas as pd

# ---------------------------------------------------------------
# Date: y = 1 pentru orice atac, 0 pentru trafic legitim
# ---------------------------------------------------------------
PREDICTORI = ["tcp.len", "mqtt.len", "tcp.time_delta", "mqtt.qos", "mqtt.dupflag"]

date = pd.read_csv(sys.argv[1], low_memory=False, usecols=PREDICTORI + ["target"])
y = (date.target != "legitimate").values.astype(float)
# prima coloană = 1, pentru interceptul modelului
X = np.column_stack([np.ones(len(date)), date[PREDICTORI].values])


# ---------------------------------------------------------------
# Algoritmul Firth
# ---------------------------------------------------------------
def verosimilitate_penalizata(X, y, beta):
    """Log-verosimilitatea penalizată Firth: log L(beta) + 0.5 * log det I(beta).

    Întoarce și p (probabilitățile), w = p(1-p) și matricea de informație I,
    care sunt refolosite de pasul Newton.
    """
    eta = np.clip(X @ beta, -500, 500)  # clip ca exp() să nu dea overflow
    p = 1 / (1 + np.exp(-eta))
    w = p * (1 - p)
    informatie = (X * w[:, None]).T @ X  # I = X' W X
    # 1e-300 evită log(0) când p ajunge exact 0 sau 1
    log_verosimilitate = np.sum(y * np.log(p + 1e-300) + (1 - y) * np.log(1 - p + 1e-300))
    penalizare = 0.5 * np.linalg.slogdet(informatie)[1]
    return log_verosimilitate + penalizare, p, w, informatie


def ajusteaza_firth(X, y, coef_fixat=None, max_iteratii=200):
    """Estimează coeficienții Firth prin Newton-Raphson cu înjumătățirea pasului.

    coef_fixat: indicele unui coeficient ținut la 0 (pentru modelul restrâns
    din testul Chisq). Întoarce (coeficienți, erori standard, log-verosimilitate
    penalizată).

    Pașii unei iterații:
      1. calculăm p, W și I = X'WX în punctul curent;
      2. h = diagonala matricei hat H = W^(1/2) X I^-1 X' W^(1/2);
      3. scorul modificat Firth: U = X' (y - p + h (0.5 - p));
      4. pasul Newton: I^-1 U (doar pe coeficienții liberi);
      5. înjumătățim pasul până când verosimilitatea penalizată nu scade.
    """
    beta = np.zeros(X.shape[1])
    liber = np.ones(X.shape[1], bool)
    if coef_fixat is not None:
        liber[coef_fixat] = False  # rămâne la valoarea de start, 0

```

```
    for _ in range(max_iteratii):
        L, p, w, informatie = verosimilitate_penalizata(X, y, beta)
        informatie_inv = np.linalg.inv(informatie)
        h = w * np.einsum("ij,jk,ik->i", X, informatie_inv, X)
        scor = X.T @ (y - p + h * (0.5 - p))
        informatie_libera = informatie[np.ix_(liber, liber)]
        pas = np.linalg.solve(informatie_libera, scor[liber])

        factor = 1.0
        while True:
            beta_nou = beta.copy()
            beta_nou[liber] += factor * pas
            if verosimilitate_penalizata(X, y, beta_nou)[0] >= L - 1e-9 or factor < 1e-4:
                break
            factor /= 2
        beta = beta_nou

        if np.max(np.abs(factor * pas)) < 1e-8:  # convergență
            break

    L, p, w, informatie = verosimilitate_penalizata(X, y, beta)
    erori_standard = np.sqrt(np.diag(np.linalg.inv(informatie)))
    return beta, erori_standard, L


# ---------------------------------------------------------------
# 9.5 Modelul complet: coeficienți și erori standard
# ---------------------------------------------------------------
coeficienti, erori_standard, L_complet = ajusteaza_firth(X, y)
print("coef", np.round(coeficienti, 4))
print("se", np.round(erori_standard, 4))

# ---------------------------------------------------------------
# 9.5 Chisq pe predictor: 2 * (L complet - L fără predictor)
# ---------------------------------------------------------------
# Modelul restrâns ține coeficientul j la 0 (j = 0 este interceptul, deci pornim de la 1).
for j, nume in enumerate(PREDICTORI, 1):
    _, _, L_restrans = ajusteaza_firth(X, y, coef_fixat=j)
    print(nume, "chisq", round(2 * (L_complet - L_restrans), 2))
```

05_cap8-9_cook_durbin_watson.py

```python
"""
Distanța Cook, reziduuri deviance, Durbin-Watson și verificarea câmpurilor „0”
(tema MQTTset, cap. 8 și 9).

Reproduce:
  - cap. 9.3: distanța Cook pentru regresia logistică atac/legitim
    (maximul, câte puncte trec de pragurile 4/n și 0,5) și reziduurile deviance;
  - cap. 8:   testul Durbin-Watson pentru cele două regresii liniare;
  - verificarea că valorile „0” din câmpurile MQTT înseamnă „câmp lipsă”
    (pachete fără MQTT), nu valoarea zero.

Intrare:  CSV-ul MQTTset și calea fișierului .npy de ieșire.
Ieșire:   text la consolă + fișierul .npy cu distanța Cook pentru fiecare pachet
          (folosit de 08_grafice_restul.py).

Rulare:
    python 05_cap8-9_cook_durbin_watson.py ../mqttdataset_reduced.csv.zip cook.npy
"""
import sys

import numpy as np
import pandas as pd
import statsmodels.api as sm
from statsmodels.stats.stattools import durbin_watson

date = pd.read_csv(sys.argv[1], low_memory=False)
fisier_cook = sys.argv[2]

# ---------------------------------------------------------------
# 9.3 Distanța Cook pentru regresia logistică
# ---------------------------------------------------------------
y = (date.target != "legitimate").astype(int).values
X = sm.add_constant(date[["tcp.len", "tcp.time_delta", "mqtt.len"]]).values
model_logistic = sm.GLM(y, X, family=sm.families.Binomial()).fit()

# clip: p exact 0 sau 1 ar da împărțire la zero mai jos
p = np.clip(model_logistic.predict(), 1e-12, 1 - 1e-12)
w = p * (1 - p)

# levierul h = diagonala matricei hat W^(1/2) X (X'WX)^-1 X' W^(1/2)
informatie_inv = np.linalg.inv((X * w[:, None]).T @ X)
levier = w * np.einsum("ij,jk,ik->i", X, informatie_inv, X)

# Cook pentru GLM: D_i = r_i^2 * h_i / (k * (1 - h_i)^2), cu r_i reziduul Pearson
reziduu_pearson = (y - p) / np.sqrt(w)
k = X.shape[1]  # numărul de parametri (cu intercept)
cook = reziduu_pearson**2 * levier / (k * (1 - levier) ** 2)

# Praguri uzuale: 4/n (regulă practică, depinde de mărimea setului) și 0,5 (influență mare)
n = len(y)
prag_4_n = 4 / n
print("cook max", cook.max(),
      "argmax row", cook.argmax() + 1,
      "n>4/n", (cook > prag_4_n).sum(), (cook > prag_4_n).mean(),
      "n>0.5", (cook > 0.5).sum(),
      "4/n", prag_4_n)

reziduu_deviance = model_logistic.resid_deviance
print("reziduuri deviance: max", np.abs(reziduu_deviance).max(),
      ">2", (np.abs(reziduu_deviance) > 2).sum(),
      ">3", (np.abs(reziduu_deviance) > 3).sum())

print("top5", np.sort(cook)[-5:])
np.save(fisier_cook, cook)

# ---------------------------------------------------------------
# Cap. 8 Durbin-Watson pentru cele două regresii liniare
# ---------------------------------------------------------------
# Regresia 1: mqtt.len ~ tcp.len; regresia 2: tcp.time_delta ~ tcp.len + mqtt.len
regresia1 = sm.OLS(date["mqtt.len"], sm.add_constant(date["tcp.len"])).fit()
regresia2 = sm.OLS(date["tcp.time_delta"], sm.add_constant(date[["tcp.len", "mqtt.len"]])).fit()
```

```
print("DW1", durbin_watson(regresia1.resid), "DW2", durbin_watson(regresia2.resid))

# ------------------------------------------------------------
# Verificarea câmpurilor „0”: apar doar la pachetele fără MQTT (msgtype 0)?
# ------------------------------------------------------------
# astype(str) separă „0” (câmp lipsă) de „0x00000000” (valoare reală)
print(pd.crosstab(date["mqtt.conack.flags"].astype(str), date["mqtt.msgtype"]))
print(pd.crosstab(date["mqtt.protoname"].astype(str), date["mqtt.msgtype"]))

fara_mqtt = date[date["mqtt.msgtype"] == 0]
print("msgtype0: hdrflags", fara_mqtt["mqtt.hdrflags"].astype(str).unique()[:5],
      "mqtt.len max", fara_mqtt["mqtt.len"].max())

print(pd.crosstab(date["mqtt.kalive"], date.target))
```

06_cap11_reguli_asociere.py

```python
"""
Reguli de asociere cu Apriori (tema MQTTset, cap. 11).

Reproduce:
  - tranzacțiile (fiecare pachet devine un set de itemi „atribut=valoare”);
  - itemseturile frecvente (support minim 5%) și cele 308 reguli după eliminarea
    regulilor redundante;
  - regulile care prezic fiecare tip de atac (support minim 300 / 150 pachete)
    și primele 3 reguli pe atac.

Intrare:  CSV-ul MQTTset; opțional, calea unui CSV de ieșire.
Ieșire:   text la consolă; dacă e dat al doilea argument, scrie în el regulile
          pe atacuri (folosit de graficul interactiv de pe site).

Rulare:
    python 06_cap11_reguli_asociere.py ../mqttdataset_reduced.csv.zip [reguli_atacuri.csv]

Notă: ordinea în care se afișează elementele unui frozenset depinde de
PYTHONHASHSEED; pentru o ieșire identică între rulări folosiți PYTHONHASHSEED=0.
"""
import sys

import pandas as pd
from mlxtend.frequent_patterns import apriori, association_rules

date = pd.read_csv(sys.argv[1], low_memory=False)

# ---------------------------------------------------------------
# 11.1 Tranzacțiile: discretizarea atributelor
# ---------------------------------------------------------------
FLAGURI_TCP = {
    "0x00000010": "ACK",
    "0x00000018": "PSH+ACK",
    "0x00000011": "FIN+ACK",
    "0x00000002": "SYN",
    "0x00000012": "SYN+ACK",
    "0x00000004": "RST",
    "0x00000019": "FIN+PSH+ACK",
    "0x00000014": "RST+ACK",
}
TIPURI_MESAJ_MQTT = {
    0: "fara",  # pachet TCP fără MQTT
    1: "CONNECT",
    2: "CONNACK",
    3: "PUBLISH",
    4: "PUBACK",
    5: "PUBREC",
    8: "SUBSCRIBE",
    9: "SUBACK",
    12: "PINGREQ",
    13: "PINGRESP",
    14: "DISCONNECT",
}

tranzactii = pd.DataFrame({
    "flags": date["tcp.flags"].map(FLAGURI_TCP),
    "msgtype": date["mqtt.msgtype"].astype(int).map(TIPURI_MESAJ_MQTT),
    "qos": date["mqtt.qos"].astype(int),
    "dupflag": date["mqtt.dupflag"].astype(int),
    # lungimea TCP: 0 = fără date, 1460 = MSS-ul obișnuit pe Ethernet
    "len": pd.cut(date["tcp.len"], [-1, 0, 20, 1460, 1e9],
                  labels=["0", "1-20", "21-1460", "1461+"]),
    # tcp.time_delta pe sferturi: limitele sunt cuartilele (Q1 iese 33% din cauza valorilor egale)
    "delta": pd.cut(date["tcp.time_delta"], [-1, 3e-6, 2.6e-5, 1.7e-4, 1e9],
                    labels=["Q1", "Q2", "Q3", "Q4"]),
    "target": date.target,
})
print(tranzactii.head(5).to_string())
print(tranzactii.delta.value_counts(normalize=True).round(3).to_dict())

# matricea one-hot: o coloană booleană pentru fiecare item „atribut=valoare”
```

```
itemi = pd.get_dummies(tranzactii.astype(str), prefix_sep="=")
print("items", itemi.shape[1])


# ---------------------------------------------------------------
# Funcții ajutătoare
# ---------------------------------------------------------------
def singurul_item(itemset):
    """Elementul unui frozenset cu un singur item (partea dreaptă a regulii)."""
    return next(iter(itemset))


def reguli_cu_un_item_in_dreapta(itemseturi, confidenta_minima):
    # Păstrăm doar reguli cu un singur item în dreapta, ca fiecare regulă să se citească
    # „aceste valori → o valoare” (decizia din 11.2).
    reguli = association_rules(itemseturi, metric="confidence", min_threshold=confidenta_minima)
    return reguli[reguli.consequents.apply(len) == 1]


def elimina_redundante(reguli):
    """Elimină regulile redundante.

    O regulă A => c e redundantă dacă există o regulă mai generală B => c
    (B submulțime strictă a lui A) cu confidența cel puțin la fel de mare:
    itemii în plus din A nu aduc nimic.
    """
    pastreaza = []
    for _, regula in reguli.iterrows():
        mai_generale = reguli[(reguli.consequents == regula.consequents)
                              & (reguli.antecedents.apply(lambda x: x < regula.antecedents))]
        pastreaza.append(len(mai_generale) == 0 or regula.confidence > mai_generale.confidence.max())
    return reguli[pastreaza]


# ---------------------------------------------------------------
# 11.2 Apriori pe tot setul: support >= 5%, confidență >= 0,6
# ---------------------------------------------------------------
frecvente = apriori(itemi, min_support=0.05, use_colnames=True)
frecvente["k"] = frecvente.itemsets.apply(len)
print("frequent per k", frecvente.k.value_counts().sort_index().to_dict())

reguli = reguli_cu_un_item_in_dreapta(frecvente, 0.6)
print("rules", len(reguli))

# n = numărul total de itemi din regulă (stânga + cel din dreapta)
reguli_finale = elimina_redundante(reguli)
reguli_finale = reguli_finale.assign(n=reguli_finale.antecedents.apply(len) + 1)
print("pruned", len(reguli_finale),
      reguli_finale.n.value_counts().sort_index().to_dict(),
      reguli_finale.support.min(), reguli_finale.support.max(),
      reguli_finale.lift.min(), reguli_finale.lift.max(), reguli_finale.lift.median())

cu_clasa_in_dreapta = reguli_finale[reguli_finale.consequents.apply(lambda x: singurul_item(x).startswith("t
arget"))]
print("RHS classes", cu_clasa_in_dreapta.consequents.apply(singurul_item).value_counts().to_dict())

top10_lift = reguli_finale.sort_values("lift", ascending=False).head(10)
print(top10_lift[["antecedents", "consequents", "support", "confidence", "lift"]].to_string())

# ---------------------------------------------------------------
# 11.3 Reguli pe atacuri
# ---------------------------------------------------------------
# Atacurile rare (ex. flood) nu ating 5%, deci pragul de support se dă
# în număr absolut de pachete (300, apoi 150).
ATACURI = ["target=" + atac for atac in ["dos", "bruteforce", "malformed", "slowite", "flood"]]

for pachete_minime in [300, 150]:
    frecvente_atac = apriori(itemi, min_support=pachete_minime / len(itemi), use_colnames=True, max_len=7)
    reguli_atac = reguli_cu_un_item_in_dreapta(frecvente_atac, 0.5)
    # dreapta = un atac, stânga = doar atribute ale pachetului (fără clasă)
    reguli_atac = reguli_atac[
        reguli_atac.consequents.apply(lambda x: singurul_item(x) in ATACURI)
        & reguli_atac.antecedents.apply(lambda x: not any(item.startswith("target") for item in x))
    ]
```

```
    reguli_atac = elimina_redundante(reguli_atac)
    print(pachete_minime, reguli_atac.consequents.apply(singurul_item).value_counts().to_dict())

# De aici se folosesc regulile de la ultimul prag (150 de pachete).
if len(sys.argv) > 2:
    pd.DataFrame({
        "stanga": reguli_atac.antecedents.apply(lambda x: " & ".join(sorted(x))),
        "atac": reguli_atac.consequents.apply(lambda x: singurul_item(x)[len("target="):]),
        "pachete": (reguli_atac.support * len(itemi)).round().astype(int),
        "support": reguli_atac.support,
        "confidence": reguli_atac.confidence,
        "lift": reguli_atac.lift,
    }).to_csv(sys.argv[2], index=False)

# primele 3 reguli pe atac (după confidență, apoi support)
for atac in ATACURI:
    reguli_atacului = reguli_atac[reguli_atac.consequents.apply(lambda x: singurul_item(x) == atac)]
    primele3 = reguli_atacului.sort_values(["confidence", "support"], ascending=False).head(3)
    for _, regula in primele3.iterrows():
        print(atac, sorted(regula.antecedents),
              round(regula.support * len(itemi)),
              round(regula.support, 4),
              round(regula.confidence, 3),
              round(regula.lift, 1))
```

07_grafice_corelatie.py

```python
"""
Graficele de corelație (tema MQTTset, cap. 7.1-7.3).

Reproduce:
  - image40.png: harta corelațiilor Spearman pe tot setul;
  - image42.png ... image47.png: aceeași hartă pe fiecare clasă
    (legitimate, dos, bruteforce, malformed, slowite, flood);
  - image41.png: trei diagrame de dispersie pe ranguri, cu dreapta de regresie
    și R Spearman; la consolă se afișează R și p pentru fiecare pereche.

Intrare:  CSV-ul MQTTset și folderul de ieșire (trebuie să existe).
Ieșire:   fișierele PNG de mai sus + text la consolă.

Rulare:
    python 07_grafice_corelatie.py ../mqttdataset_reduced.csv.zip grafice
"""
import sys

import matplotlib
import numpy as np
import pandas as pd

matplotlib.use("Agg")  # fără fereastră, doar salvare în fișiere
import matplotlib.pyplot as plt
from matplotlib.ticker import FuncFormatter
from scipy import stats

date = pd.read_csv(sys.argv[1], low_memory=False)
folder_iesire = sys.argv[2]

VARIABILE = [
    "tcp.time_delta", "tcp.len", "mqtt.len", "mqtt.kalive", "mqtt.qos",
    "mqtt.dupflag", "mqtt.retain", "mqtt.conflag.cleansess", "mqtt.conflag.passwd",
    "mqtt.conflag.uname", "mqtt.proto_len", "mqtt.ver", "mqtt.conack.val",
]


def format_ro(valoare, zecimale=2):
    """Număr în format românesc: virgulă zecimală, minus tipografic, fără „-0,00”."""
    if round(valoare, zecimale) == 0:
        valoare = 0.0
    return f"{valoare:.{zecimale}f}".replace(".", ",").replace("-", "−")


# celulele NaN (corelație imposibilă) apar gri
paleta = plt.get_cmap("RdBu").copy()
paleta.set_bad("#dddddd")


def harta_corelatie(subset, titlu, nume_fisier):
    """Harta corelațiilor Spearman dintre VARIABILE, cu valoarea scrisă în fiecare celulă."""
    # o variabilă constantă în subset dă NaN (nu are ranguri diferite)
    corelatii = subset[VARIABILE].corr("spearman").values
    fig, ax = plt.subplots(figsize=(11.7, 9.88), dpi=100)
    imagine = ax.imshow(np.ma.masked_invalid(corelatii), cmap=paleta, vmin=-1, vmax=1)

    for i in range(len(VARIABILE)):
        for j in range(len(VARIABILE)):
            if not np.isnan(corelatii[i, j]):
                # text alb pe celulele închise la culoare, negru în rest
                culoare_text = "white" if abs(corelatii[i, j]) > 0.5 else "black"
                ax.text(j, i, format_ro(corelatii[i, j]),
                        ha="center", va="center", fontsize=7.5, color=culoare_text)

    ax.set_xticks(range(len(VARIABILE)))
    ax.set_xticklabels(VARIABILE, rotation=90, fontsize=9)
    ax.set_yticks(range(len(VARIABILE)))
    ax.set_yticklabels(VARIABILE, fontsize=9)
    bara_culori = fig.colorbar(imagine, ax=ax, shrink=0.8)
    bara_culori.ax.yaxis.set_major_formatter(FuncFormatter(lambda v, _: format_ro(v)))
    ax.set_title(titlu, fontsize=10)
```

```
    fig.tight_layout()
    fig.savefig(f"{folder_iesire}/{nume_fisier}")
    plt.close(fig)


# ---------------------------------------------------------------
# 7.1 Corelația Spearman pe tot setul (image40)
# ---------------------------------------------------------------
harta_corelatie(date,
                "Corelația Spearman, variabile numerice, ordinale și binare, tot setul (gri = nu se poate ca
lcula)",
                "image40.png")

# ---------------------------------------------------------------
# 7.3 Corelația Spearman pe fiecare clasă (image42-image47)
# ---------------------------------------------------------------
CLASE = ["legitimate", "dos", "bruteforce", "malformed", "slowite", "flood"]
for numar_imagine, clasa in zip(range(42, 48), CLASE):
    harta_corelatie(date[date.target == clasa],
                    f"Corelația Spearman, {clasa} (gri = variabilă constantă în această clasă)",
                    f"image{numar_imagine}.png")

# ---------------------------------------------------------------
# 7.2 Diagrame de dispersie pe ranguri (image41)
# ---------------------------------------------------------------
# 8000 de pachete ajung pentru grafic; R și p se calculează pe tot setul.
esantion = date.sample(8000, random_state=42)
PERECHI = [("tcp.len", "mqtt.len"), ("tcp.time_delta", "tcp.len"), ("tcp.time_delta", "mqtt.len")]

fig, axe = plt.subplots(1, 3, figsize=(15.6, 4.94), dpi=100)
for ax, (var_x, var_y) in zip(axe, PERECHI):
    R, p = stats.spearmanr(date[var_x], date[var_y])

    # Spearman = Pearson pe ranguri, deci desenăm rangurile, nu valorile
    rang_x = esantion[var_x].rank()
    rang_y = esantion[var_y].rank()
    ax.scatter(rang_x, rang_y, s=4, c="black", alpha=0.3)

    panta, intercept = np.polyfit(rang_x, rang_y, 1)
    capete_x = np.array([rang_x.min(), rang_x.max()])
    ax.plot(capete_x, intercept + panta * capete_x, color="#3355aa")

    ax.set_xlabel(f"{var_x} (rang)")
    ax.set_ylabel(f"{var_y} (rang)")
    ax.set_title(f"R (Spearman) = {format_ro(R)}, p < 0,001", fontsize=10)
    print(var_x, var_y, R, p)

fig.tight_layout()
fig.savefig(f"{folder_iesire}/image41.png")
plt.close(fig)
```

08_grafice_restul.py

```python
"""
Graficele din raport care nu sunt produse de celelalte scripturi:
cap. 5 (distribuțiile variabilelor), cap. 6 (variabile pe clase),
7.4 (corelații Pearson / Spearman), cap. 8 (regresia liniară),
cap. 9 (distanța Cook, regresia logistică) și 11.4 (regulile de asociere).

Intrări:
  1. CSV-ul MQTTset redus (mqttdataset_reduced.csv)
  2. folderul în care se scriu imaginile
  3. fișierul .npy cu distanțele Cook, scris de 05_cap8-9_cook_durbin_watson.py

Ieșiri:
  imageN.png în folderul de ieșire (N = numărul figurii din document);
  la ecran: testele Mann-Whitney din cap. 6 și P(atac) la tcp.time_delta = 0.

Rulare:
  python 08_grafice_restul.py mqttdataset_reduced.csv folder_grafice cook.npy
"""
import sys

import matplotlib
import numpy as np
import pandas as pd
import statsmodels.api as sm

matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.ticker import FuncFormatter
from mlxtend.frequent_patterns import apriori, association_rules
from scipy import stats

date = pd.read_csv(sys.argv[1], low_memory=False)
folder_iesire = sys.argv[2]
distante_cook = np.load(sys.argv[3])

date["is_attack"] = (date.target != "legitimate").astype(int)
CLASE = ["legitimate", "dos", "bruteforce", "malformed", "slowite", "flood"]
ATACURI = CLASE[1:]


# ---------------------------------------------------------------------------
# Funcții ajutătoare: formatare românească (virgulă zecimală, minus tipografic)
# ---------------------------------------------------------------------------

def format_axa(valoare, _=None):
    """Eticheta unei gradații: 0.5 -> „0,5”, -1 -> „−1”."""
    text = ("%g" % valoare).replace(".", ",")
    return text.replace("-", "−")


def format_zecimale(valoare, zecimale):
    text = f"{valoare:.{zecimale}f}"
    return text.replace(".", ",").replace("-", "−")


def virgule_pe_axe(ax, si_pe_x=True):
    """Virgulă zecimală pe axele liniare. si_pe_x=False păstrează etichetele de categorii de pe x."""
    if si_pe_x and ax.get_xscale() == "linear":
        ax.xaxis.set_major_formatter(FuncFormatter(format_axa))
    if ax.get_yscale() == "linear":
        ax.yaxis.set_major_formatter(FuncFormatter(format_axa))


def eticheta_valoare(valoare):
    """3.0 -> „3”, 65535.0 -> „65.535”; valorile hex / text rămân neschimbate."""
    if isinstance(valoare, float) and valoare.is_integer():
        return f"{int(valoare):,}".replace(",", ".")
    return str(valoare)


def salveaza(fig, nume_fisier, latime_px, inaltime_px):
```

```
    fig.set_size_inches(latime_px / 100, inaltime_px / 100)
    fig.savefig(f"{folder_iesire}/{nume_fisier}", dpi=100)
    plt.close(fig)


def stele(p):
    if p < 0.001:
        return "***"
    if p < 0.01:
        return "**"
    if p < 0.05:
        return "*"
    return "ns"


# ------------------------------------------------------------------------
# Cap. 5: distribuția fiecărei variabile (bare cu % din pachete)
# ------------------------------------------------------------------------

FIGURI_CAP5 = {
    2: "mqtt.conack.flags", 3: "mqtt.conack.val", 4: "mqtt.conflag.cleansess",
    5: "mqtt.conflag.passwd", 6: "mqtt.conflag.uname", 7: "mqtt.conflags",
    8: "mqtt.dupflag", 9: "mqtt.hdrflags", 12: "mqtt.msgtype", 13: "mqtt.proto_len",
    14: "mqtt.protoname", 15: "mqtt.qos", 16: "mqtt.retain", 17: "mqtt.ver", 18: "tcp.flags",
}


def figura_distributie(numar_figura, coloana):
    procente = date[coloana].value_counts(normalize=True) * 100
    # peste 10 valori distincte: primele 10, restul adunate în „Altele”
    if len(procente) > 10:
        procente = pd.concat([procente.iloc[:10], pd.Series({"Altele": procente.iloc[10:].sum()})])
    fig, ax = plt.subplots()
    ax.bar([eticheta_valoare(v) for v in procente.index], procente.values)
    ax.set_title(coloana)
    ax.set_ylabel("% din pachete")
    plt.setp(ax.get_xticklabels(), rotation=45, ha="right")
    virgule_pe_axe(ax, si_pe_x=False)
    fig.tight_layout()
    salveaza(fig, f"image{numar_figura}.png", 600, 420)


for numar_figura, coloana in FIGURI_CAP5.items():
    figura_distributie(numar_figura, coloana)


# ------------------------------------------------------------------------
# Cap. 6: fiecare variabilă pe clase (boxplot pentru numerice, bare pentru categorice)
# ------------------------------------------------------------------------

FIGURI_CAP6 = {
    21: "tcp.flags", 22: "tcp.time_delta", 23: "tcp.len", 24: "mqtt.hdrflags",
    25: "mqtt.msgtype", 26: "mqtt.dupflag", 27: "mqtt.qos", 28: "mqtt.retain",
    29: "mqtt.len", 30: "mqtt.conflags", 31: "mqtt.conflag.cleansess",
    32: "mqtt.conflag.passwd", 33: "mqtt.conflag.uname", 34: "mqtt.kalive",
    35: "mqtt.protoname", 36: "mqtt.proto_len", 37: "mqtt.ver",
    38: "mqtt.conack.flags", 39: "mqtt.conack.val",
}
NUMERICE = ["tcp.time_delta", "tcp.len", "mqtt.len"]
# variabile ordinale desenate ca bare, dar testate cu Kruskal-Wallis / Mann-Whitney, nu chi-pătrat
ORDINALE_CU_BARE = ["mqtt.qos", "mqtt.kalive"]
legitime = date[date.target == "legitimate"]


def mann_whitney_atac_vs_normal(coloana):
    """Stelele testului Mann-Whitney pentru fiecare atac comparat cu traficul normal."""
    rezultat = {}
    for atac in ATACURI:
        normal = legitime[coloana].values
        din_atac = date[date.target == atac][coloana].values
        # o singură valoare în ambele grupuri: testul nu are sens
        if np.unique(np.r_[normal, din_atac]).size == 1:
            rezultat[atac] = "-"
        else:
```

```
            rezultat[atac] = stele(stats.mannwhitneyu(normal, din_atac).pvalue)
    return rezultat


def boxplot_pe_clase(ax, coloana):
    valori = [date[date.target == clasa][coloana].values for clasa in CLASE]
    if coloana == "tcp.time_delta":
        # scara log nu poate afișa 0, așa că 0 este desenat ca 1e-6 s
        valori = [np.where(v == 0, 1e-6, v) for v in valori]
        ax.set_yscale("log")
        nota = "; 0 afișat ca 1e-06 s"
    else:
        ax.set_yscale("symlog", linthresh=1)
        nota = ""
    ax.boxplot(valori, showfliers=False, widths=0.5)
    ax.set_xticks(range(1, 7))
    ax.set_xticklabels(CLASE)
    sus = ax.get_ylim()[1]
    teste = mann_whitney_atac_vs_normal(coloana)
    for pozitie, atac in enumerate(ATACURI, 2):
        ax.text(pozitie, sus, teste[atac], ha="center", va="bottom")
    # loc deasupra cutiilor pentru stele (axele sunt logaritmice, deci înmulțim)
    ax.set_ylim(top=sus * (30 if coloana == "tcp.time_delta" else 8))
    ax.set_title(f"{coloana}  (Kruskal-Wallis ***; stele: atac vs normal{nota})", fontsize=9)
    print(coloana, teste)


def bare_pe_clase(ax, coloana):
    if date[coloana].dtype != object:
        valori = date[coloana].map(eticheta_valoare)
    else:
        valori = date[coloana].astype(str)
    # cele mai frecvente 6 valori, restul „Altele”
    frecvente = valori.value_counts().index[:6]
    valori = valori.where(valori.isin(frecvente), "Altele")
    if coloana == "mqtt.kalive":
        grupe_kalive = {0: "0", 60: "60", 65535: "65.535"}
        valori = date[coloana].map(lambda v: grupe_kalive.get(v, "altele (1, 2, 3, 234)"))
    procente = pd.crosstab(valori, date.target, normalize="columns")[CLASE] * 100
    # ordinea în legendă: valorile scurte întâi, „Altele” la final
    ordine = [v for v in sorted(procente.index, key=lambda v: (v.startswith(("Altele", "altele")), len(v),
v))]
    procente = procente.loc[ordine]
    numar_valori = len(procente)
    latime_bara = 0.8 / numar_valori
    for i, (valoare, rand) in enumerate(procente.iterrows()):
        deplasare = (i - (numar_valori - 1) / 2) * latime_bara
        ax.bar(np.arange(6) + deplasare, rand.values, latime_bara, label=valoare)
    ax.set_xticks(range(6))
    ax.set_xticklabels(CLASE)
    ax.set_ylabel("% din clasă")
    if coloana in ORDINALE_CU_BARE:
        teste = mann_whitney_atac_vs_normal(coloana)
        ax.set_ylim(0, 112)
        for pozitie, atac in enumerate(ATACURI, 1):
            ax.text(pozitie, 104, teste[atac], ha="center")
        ax.set_title(f"{coloana}  (Kruskal-Wallis ***; stele: atac vs normal)", fontsize=9)
        print(coloana, teste)
    else:
        ax.set_title(f"{coloana}  (chi-pătrat ***)", fontsize=9)
    ax.legend(title="valoare", fontsize=7, title_fontsize=7, loc="upper left", bbox_to_anchor=(1.01, 1))


for numar_figura, coloana in FIGURI_CAP6.items():
    fig, ax = plt.subplots()
    if coloana in NUMERICE:
        boxplot_pe_clase(ax, coloana)
    else:
        bare_pe_clase(ax, coloana)
    plt.setp(ax.get_xticklabels(), rotation=30, ha="right")
    virgule_pe_axe(ax, si_pe_x=False)
    fig.set_size_inches(7.8, 5.04)
    fig.tight_layout()
    salveaza(fig, f"image{numar_figura}.png", 780, 504)
```

```
219
220
221 # ------------------------------------------------------------------------
222 # 7.4: matricile de corelație Pearson și Spearman (figura 48)
223 # ------------------------------------------------------------------------
224
225 def figura_corelatii():
226     variabile = ["tcp.len", "mqtt.len", "tcp.time_delta", "mqtt.qos", "mqtt.dupflag", "is_attack"]
227     fig, axs = plt.subplots(1, 2, figsize=(12.53, 5.87))
228     for ax, metoda in zip(axs, ["pearson", "spearman"]):
229         corelatii = date[variabile].corr(metoda).values
230         imagine = ax.imshow(corelatii, cmap="RdBu", vmin=-1, vmax=1)
231         for i in range(6):
232             for j in range(6):
233                 culoare_text = "white" if abs(corelatii[i, j]) > 0.5 else "black"
234                 ax.text(j, i, format_zecimale(corelatii[i, j], 2), ha="center", va="center",
235                         fontsize=8, color=culoare_text)
236         ax.set_xticks(range(6))
237         ax.set_xticklabels(variabile, rotation=40, ha="right", fontsize=8)
238         ax.set_yticks(range(6))
239         # numele variabilelor doar pe matricea din stânga
240         ax.set_yticklabels(variabile if metoda == "pearson" else [], fontsize=8)
241         ax.set_title(metoda.capitalize())
242     bara_culori = fig.colorbar(imagine, ax=axs, shrink=0.85)
243     bara_culori.ax.yaxis.set_major_formatter(FuncFormatter(lambda v, _: format_zecimale(v, 2)))
244     salveaza(fig, "image48.png", 1253, 587)
245
246
247 figura_corelatii()
248
249
250 # ------------------------------------------------------------------------
251 # Cap. 8: regresia liniară (figurile 49-52)
252 # ------------------------------------------------------------------------
253
254 # 8000 de pachete pentru norii de puncte; dreptele se calculează pe toate datele
255 esantion = date.sample(8000, random_state=42)
256
257 model_brut = sm.OLS(date["mqtt.len"], sm.add_constant(date["tcp.len"])).fit()
258 logaritmi = np.log1p(date[["tcp.len", "mqtt.len"]])
259 model_log = sm.OLS(logaritmi["mqtt.len"], sm.add_constant(logaritmi["tcp.len"])).fit()
260
261
262 def dreapta(ax, model, x_min, x_max):
263     xx = np.array([x_min, x_max])
264     ax.plot(xx, model.params.iloc[0] + model.params.iloc[1] * xx, "r")
265
266
267 def figura_mqtt_len_din_tcp_len():
268     fig, axs = plt.subplots(1, 2)
269
270     axs[0].scatter(esantion["tcp.len"], esantion["mqtt.len"], s=4, alpha=0.4)
271     dreapta(axs[0], model_brut, 0, date["tcp.len"].max())
272     axs[0].set_title(f"Valori brute, R² = {format_zecimale(model_brut.rsquared, 3)}")
273     axs[0].set_xlabel("tcp.len")
274     axs[0].set_ylabel("mqtt.len")
275
276     esantion_log = np.log1p(esantion[["tcp.len", "mqtt.len"]])
277     axs[1].scatter(esantion_log["tcp.len"], esantion_log["mqtt.len"], s=4, alpha=0.4)
278     dreapta(axs[1], model_log, 0, logaritmi["tcp.len"].max())
279     axs[1].set_title(f"După logaritm, R² = {format_zecimale(model_log.rsquared, 3)}")
280     axs[1].set_xlabel("log(1 + tcp.len)")
281     axs[1].set_ylabel("log(1 + mqtt.len)")
282
283     for ax in axs:
284         virgule_pe_axe(ax)
285     fig.set_size_inches(14.3, 5.46)
286     fig.tight_layout()
287     salveaza(fig, "image49.png", 1430, 546)
288
289
290 def reziduuri(ax_puncte, ax_histograma, prezise, reziduu, sufix_titlu=""):
291     """Reziduuri față de predicție + histograma lor cu curba normală (media și abaterea reziduurilor)."""
292     i = esantion.index
```

```
    ax_puncte.scatter(prezise[i], reziduu[i], s=4, alpha=0.4)
    ax_puncte.axhline(0, color="r")
    ax_puncte.set_title("Reziduuri față de predicție" + sufix_titlu)
    ax_puncte.set_xlabel("valoare prezisă")
    ax_puncte.set_ylabel("reziduu")

    # fără 1% din fiecare capăt, altfel câteva valori extreme strivesc histograma
    cuantile = reziduu.quantile([.01, .99])
    centrale = reziduu[(reziduu >= cuantile.iloc[0]) & (reziduu <= cuantile.iloc[1])]
    _, margini, _ = ax_histograma.hist(centrale, bins=50)
    xs = np.linspace(margini[0], margini[-1], 300)
    latime_bin = margini[1] - margini[0]
    curba = len(centrale) * latime_bin * stats.norm.pdf(xs, centrale.mean(), centrale.std())
    ax_histograma.plot(xs, curba, "r", lw=2, label="curba normală")
    ax_histograma.set_title("Histograma reziduurilor, 1% până la 99%" + sufix_titlu)
    ax_histograma.set_xlabel("reziduu")
    ax_histograma.legend(frameon=False)
    virgule_pe_axe(ax_puncte)
    virgule_pe_axe(ax_histograma)


def figura_reziduuri_mqtt_len():
    fig, axs = plt.subplots(2, 2)
    reziduuri(*axs[0], model_brut.fittedvalues, model_brut.resid, " (brute)")
    reziduuri(*axs[1], model_log.fittedvalues, model_log.resid, " (logaritm)")
    fig.set_size_inches(14.3, 9.1)
    fig.tight_layout()
    salveaza(fig, "image50.png", 1430, 910)


def figura_reziduuri_time_delta():
    model_timp = sm.OLS(date["tcp.time_delta"], sm.add_constant(date[["tcp.len", "mqtt.len"]])).fit()
    fig, axs = plt.subplots(1, 2)
    reziduuri(*axs, model_timp.fittedvalues, model_timp.resid)
    fig.set_size_inches(14.3, 5.46)
    fig.tight_layout()
    salveaza(fig, "image51.png", 1430, 546)


def figura_regresie_pe_clase():
    fig, axs = plt.subplots(2, 3)
    for ax, clasa in zip(axs.flat, CLASE):
        din_clasa = date[date.target == clasa]
        model = sm.OLS(din_clasa["mqtt.len"], sm.add_constant(din_clasa["tcp.len"])).fit()
        puncte = din_clasa if len(din_clasa) < 3000 else din_clasa.sample(3000, random_state=42)
        ax.scatter(puncte["tcp.len"], puncte["mqtt.len"], s=4, alpha=0.4)
        dreapta(ax, model, din_clasa["tcp.len"].min(), din_clasa["tcp.len"].max())
        panta = format_zecimale(model.params.iloc[1], 3)
        ax.set_title(f"{clasa}: panta {panta}, R² = {format_zecimale(model.rsquared, 3)}", fontsize=9)
        ax.set_xlabel("tcp.len", fontsize=8)
        ax.set_ylabel("mqtt.len", fontsize=8)
        ax.tick_params(labelsize=7)
        virgule_pe_axe(ax)
    fig.set_size_inches(16.9, 9.1)
    fig.tight_layout()
    salveaza(fig, "image52.png", 1690, 910)


figura_mqtt_len_din_tcp_len()
figura_reziduuri_mqtt_len()
figura_reziduuri_time_delta()
figura_regresie_pe_clase()


# ---------------------------------------------------------------------------
# Cap. 9: distanța Cook, curba P(atac) și probabilitățile prezise (figurile 53, 56, 57)
# ---------------------------------------------------------------------------

def figura_cook():
    n = len(distante_cook)
    fig, ax = plt.subplots()
    ax.vlines(np.arange(1, n + 1), 0, distante_cook, lw=0.5, alpha=0.6)
    # 4/n este pragul uzual pentru „influent”; 0,5 sau 1 sunt pragurile critice
    ax.axhline(4 / n, color="r", ls="--", lw=0.8, label=f"4/n = {format_zecimale(4 / n * 1e5, 1)} · 10⁻⁵")
```

```
    ax.set_title("Distanța Cook, modelul fără separare (maxim " + format_zecimale(distante_cook.max(), 4)
                 + ", pragul critic 0,5 sau 1)", fontsize=10)
    ax.set_xlabel("numărul pachetului (rândul din fișier)")
    ax.set_ylabel("distanța Cook")
    ax.legend(fontsize=8)
    virgule_pe_axe(ax)
    ax.xaxis.set_major_formatter(FuncFormatter(lambda v, _: f"{int(v):,}".replace(",", ".")))
    fig.set_size_inches(13, 5.2)
    fig.tight_layout()
    salveaza(fig, "image53.png", 1300, 520)


# modelul logistic final: is_attack ~ tcp.len + tcp.time_delta + mqtt.len
model_logistic = sm.GLM(date.is_attack, sm.add_constant(date[["tcp.len", "tcp.time_delta", "mqtt.len"]]),
                        family=sm.families.Binomial()).fit()


def figura_curba_probabilitate():
    coef = model_logistic.params
    timp = np.linspace(0, 2, 200)
    # tcp.len și mqtt.len fixate la valorile tipice (12 și 10)
    logit = coef.iloc[0] + coef["tcp.len"] * 12 + coef["tcp.time_delta"] * timp + coef["mqtt.len"] * 10
    probabilitate = 1 / (1 + np.exp(-logit))
    fig, ax = plt.subplots()
    ax.plot(timp, probabilitate, color="#d62728", lw=2)
    ax.axhline(0.5, color="gray", ls=":")
    ax.set_ylim(0, 1)
    ax.set_title("Probabilitatea de atac prezisă de modelul final\n"
                 "(tcp.len = 12 și mqtt.len = 10, valorile tipice; doar timpul variază)", fontsize=10)
    ax.set_xlabel("tcp.time_delta (secunde)")
    ax.set_ylabel("P(atac)")
    virgule_pe_axe(ax)
    fig.set_size_inches(10.5, 4.8)
    fig.tight_layout()
    salveaza(fig, "image56.png", 1050, 480)
    print("curve p0", probabilitate[0])


def figura_probabilitati_prezise():
    prezise = model_logistic.predict()
    fig, ax = plt.subplots()
    intervale = np.linspace(0, 1, 51)
    ax.hist(prezise[date.is_attack == 0], bins=intervale, alpha=0.6, label="normal (0)")
    ax.hist(prezise[date.is_attack == 1], bins=intervale, alpha=0.6, color="#e15759", label="atac (1)")
    ax.set_yscale("log")
    ax.set_xlabel("probabilitatea de atac prezisă")
    ax.set_ylabel("pachete (scară log)")
    ax.set_title("Ce scoate modelul final pentru fiecare pachet, separat pe grupul real", fontsize=10)
    ax.legend()
    virgule_pe_axe(ax)
    fig.set_size_inches(10.5, 4.8)
    fig.tight_layout()
    salveaza(fig, "image57.png", 1050, 480)


figura_cook()
figura_curba_probabilitate()
figura_probabilitati_prezise()


# ---------------------------------------------------------------------------
# 11.4: regulile de asociere, support față de confidence (figura 64)
# ---------------------------------------------------------------------------

NUME_FLAGURI_TCP = {
    "0x00000010": "ACK", "0x00000018": "PSH+ACK", "0x00000011": "FIN+ACK", "0x00000002": "SYN",
    "0x00000012": "SYN+ACK", "0x00000004": "RST", "0x00000019": "FIN+PSH+ACK", "0x00000014": "RST+ACK",
}
NUME_MESAJE_MQTT = {
    0: "fara", 1: "CONNECT", 2: "CONNACK", 3: "PUBLISH", 4: "PUBACK", 5: "PUBREC",
    8: "SUBSCRIBE", 9: "SUBACK", 12: "PINGREQ", 13: "PINGRESP", 14: "DISCONNECT",
}
CULORI_ATACURI = {"dos": "#e8692f", "bruteforce": "#1aae80", "malformed": "#f0a30a",
                  "slowite": "#e377a6", "flood": "#008000"}
```

```
441
442 # fiecare pachet devine o tranzacție; variabilele continue sunt împărțite în intervale
443 tranzactii = pd.DataFrame({
444     "flags": date["tcp.flags"].map(NUME_FLAGURI_TCP),
445     "msgtype": date["mqtt.msgtype"].astype(int).map(NUME_MESAJE_MQTT),
446     "qos": date["mqtt.qos"].astype(int),
447     "dupflag": date["mqtt.dupflag"].astype(int),
448     "len": pd.cut(date["tcp.len"], [-1, 0, 20, 1460, 1e9], labels=["0", "1-20", "21-1460", "1461+"]),
449     "delta": pd.cut(date["tcp.time_delta"], [-1, 3e-6, 2.6e-5, 1.7e-4, 1e9], labels=["Q1", "Q2", "Q3", "Q
4"]),
450     "target": date.target,
451 })
452 itemi = pd.get_dummies(tranzactii.astype(str), prefix_sep="=")
453
454
455 def elimina_redundante(reguli):
456     """Păstrează o regulă doar dacă are confidence mai mare decât orice regulă mai scurtă cu aceeași concluz
ie."""
457     pastreaza = []
458     for _, regula in reguli.iterrows():
459         mai_scurte = reguli[(reguli.consequents == regula.consequents)
460                             & (reguli.antecedents.apply(lambda x: x < regula.antecedents))]
461         pastreaza.append(len(mai_scurte) == 0 or regula.confidence > mai_scurte.confidence.max())
462     return reguli[pastreaza]
463
464
465 def concluzia(itemset):
466     return next(iter(itemset))
467
468
469 # rularea generală: support ≥ 5%, confidence ≥ 60%, o singură concluzie
470 reguli_generale = association_rules(apriori(itemi, min_support=0.05, use_colnames=True),
471                                     metric="confidence", min_threshold=0.6)
472 reguli_generale = elimina_redundante(reguli_generale[reguli_generale.consequents.apply(len) == 1])
473
474 # rularea pe atacuri: support de doar 150 de pachete (atacurile rare), concluzia este un atac,
475 # iar premisele nu conțin clasa
476 reguli_atacuri = association_rules(apriori(itemi, min_support=150 / len(itemi), use_colnames=True),
477                                    metric="confidence", min_threshold=0.5)
478 reguli_atacuri = reguli_atacuri[reguli_atacuri.consequents.apply(len) == 1]
479 concluzii_atac = ["target=" + atac for atac in ATACURI]
480 concluzie_e_atac = reguli_atacuri.consequents.apply(lambda x: concluzia(x) in concluzii_atac)
481 premise_fara_clasa = reguli_atacuri.antecedents.apply(lambda x: not any(i.startswith("target") for i in x))
482 reguli_atacuri = elimina_redundante(reguli_atacuri[concluzie_e_atac & premise_fara_clasa])
483
484
485 def figura_reguli():
486     fig, axs = plt.subplots(1, 2)
487
488     puncte = axs[0].scatter(reguli_generale.support, reguli_generale.confidence, c=reguli_generale.lift,
489                             cmap="Blues", s=14, edgecolors="gray", linewidths=0.3)
490     bara_culori = fig.colorbar(puncte, ax=axs[0])
491     bara_culori.set_label("lift")
492     bara_culori.ax.yaxis.set_major_formatter(FuncFormatter(format_axa))
493     axs[0].set_title(f"Rularea generală ({len(reguli_generale)} reguli)")
494     axs[0].set_xlabel("support")
495     axs[0].set_ylabel("confidence")
496
497     for atac in ATACURI:
498         ale_atacului = reguli_atacuri[reguli_atacuri.consequents.apply(lambda x: concluzia(x) == "target=" +
atac)]
499         axs[1].scatter(ale_atacului.support, ale_atacului.confidence, s=14, color=CULORI_ATACURI[atac], labe
l=atac)
500     axs[1].set_xscale("log")
501     axs[1].set_title(f"Reguli pe atacuri ({len(reguli_atacuri)} reguli)")
502     axs[1].set_xlabel("support (scară logaritmică)")
503     axs[1].set_ylabel("confidence")
504     axs[1].legend(fontsize=8)
505
506     for ax in axs:
507         virgule_pe_axe(ax)
508     fig.set_size_inches(18, 7.6)
509     fig.tight_layout()
510     salveaza(fig, "image64.png", 1800, 760)
```

```
511
512
513 figura_reguli()
```

09_cap10_arbori_ctree.R

```r
# ============================================================================
# Cap. 10: arborii de decizie cu partykit::ctree
#
# Reproduce cifrele din cap. 10: alegerea adâncimii (10.3), tabelul testelor
# din rădăcină (10.1), arborele de clasificare și frunzele lui (10.4),
# metricile pe test (10.5), verificările din 10.2 / 10.4 și arborele de
# regresie pentru tcp.len (10.6).
#
# ctree alege la fiecare nod variabila cu cel mai mic p al unui test de
# independență și desparte nodul doar dacă acest p, corectat Bonferroni,
# este sub 0,05. Așa arborele se oprește singur, fără tăiere ulterioară.
#
# Intrări:
#   1. arhiva zip cu mqttdataset_reduced.csv
#   2. (opțional) folderul de ieșire pentru CSV-uri
# Ieșiri:
#   la ecran: toate rezultatele;
#   în folder: adancime_clasificare.csv, adancime_regresie.csv (10.3) și
#   noduri_clasificare.csv, noduri_regresie.csv (pentru 10_grafice_arbori.py).
#
# Rulare (durează câteva minute; are nevoie de r-base și r-cran-partykit):
#   Rscript 09_cap10_arbori_ctree.R ../mqttdataset_reduced.csv.zip folder_grafice
# ============================================================================

suppressMessages(library(partykit))
argumente <- commandArgs(TRUE)


# ---- Citirea datelor ---------------------------------------------------------

# Coloanele hex se citesc ca text: altfel R le transformă în numere,
# iar „0” și „0x00000000” ar deveni aceeași valoare.
d <- read.csv(unz(argumente[1], "mqttdataset_reduced.csv"), check.names = FALSE,
              colClasses = c(tcp.flags = "character", mqtt.conack.flags = "character",
                             mqtt.conflags = "character", mqtt.hdrflags = "character"))
names(d) <- make.names(names(d))

# Textul și codurile (tipul mesajului, dupflag, versiunea) sunt categorii, nu numere.
for (v in c(names(d)[sapply(d, is.character)], "mqtt.msgtype", "mqtt.dupflag", "mqtt.ver")) {
  d[[v]] <- factor(d[[v]])
}
options(digits = 10)  # pragurile din print, nerotunjite
d$is_attack <- factor(ifelse(d$target == "legitimate", "normal", "atac"), c("normal", "atac"))


# ---- Împărțirea 70/30 --------------------------------------------------------

# Separat pe fiecare clasă, ca atacurile rare să aibă aceleași proporții în ambele părți.
set.seed(42)
in_constructie <- logical(nrow(d))
for (clasa in levels(d$target)) {
  randuri <- which(d$target == clasa)
  in_constructie[sample(randuri, round(0.7 * length(randuri)))] <- TRUE
}
constr <- d[in_constructie, ]
test <- d[!in_constructie, ]
cat("constructie", nrow(constr), "test", nrow(test), "\n")
print(table(constr$target))
print(table(constr$is_attack))


# ---- Setările arborilor și metricile -----------------------------------------

# quadratic + Bonferroni: testul chi-pătrat al fiecărei variabile, cu p înmulțit
#   cu numărul de variabile testate (altfel, cu 7 variabile, una ar ieși
#   „semnificativă” din întâmplare mult mai des de 5% din cazuri);
# mincriterion 0,95: se desparte doar dacă 1 - p ajustat > 0,95;
# minbucket 300: fiecare frunză are măcar 300 de pachete, ca să nu apară
#   frunze pentru câteva pachete izolate (cu 230.000 de rânduri, aproape orice
#   despărțire ar fi „semnificativă”).
setari <- function(maxdepth, testtype = "Bonferroni") {
```

```r
  ctree_control(teststat = "quadratic", testtype = testtype, mincriterion = 0.95,
                minsplit = 20, minbucket = 300, maxdepth = maxdepth)
}
X7 <- c("tcp.flags", "mqtt.msgtype", "tcp.len", "tcp.time_delta", "mqtt.qos", "mqtt.dupflag", "mqtt.ver")
formula_clasificare <- reformulate(X7, "is_attack")
formula_regresie <- tcp.len ~ target + tcp.flags + mqtt.msgtype + tcp.time_delta

# acuratețe, sensibilitate, precizie, F1; clasa pozitivă este „atac”
metrici_clasificare <- function(real, prezis) {
  tp <- sum(prezis == "atac" & real == "atac")
  fp <- sum(prezis == "atac" & real == "normal")
  fn <- sum(prezis == "normal" & real == "atac")
  sensibilitate <- tp / (tp + fn)
  precizie <- tp / (tp + fp)
  round(c(acc = mean(prezis == real), sens = sensibilitate, prec = precizie,
          f1 = 2 * sensibilitate * precizie / (sensibilitate + precizie)), 4)
}
metrici_regresie <- function(real, prezis) {
  round(c(MAE = mean(abs(real - prezis)), RMSE = sqrt(mean((real - prezis)^2)),
          R2 = 1 - sum((real - prezis)^2) / sum((real - mean(real))^2)), 4)
}

folder_iesire <- if (is.na(argumente[2])) NULL else {
  dir.create(argumente[2], showWarnings = FALSE)
  argumente[2]
}
afiseaza_si_salveaza <- function(tabel, fisier) {
  print(tabel)
  if (!is.null(folder_iesire)) write.csv(tabel, file.path(folder_iesire, fisier), row.names = FALSE)
}


# ---- 10.3 Alegerea adâncimii maxime ----------------------------------------

cat("\n10.3 maxdepth, clasificare\n")
# coloanele din CSV: acc, f1 pe construcție; acc.1, f1.1 pe test
afiseaza_si_salveaza(do.call(rbind, lapply(2:6, function(md) {
  arbore <- ctree(formula_clasificare, constr, control = setari(md))
  data.frame(maxdepth = md, frunze = width(arbore),
             t(metrici_clasificare(constr$is_attack, predict(arbore))[c(1, 4)]),
             t(metrici_clasificare(test$is_attack, predict(arbore, test))[c(1, 4)]))
})), "adancime_clasificare.csv")

cat("\n10.3 maxdepth, regresie\n")
afiseaza_si_salveaza(do.call(rbind, lapply(2:6, function(md) {
  arbore <- ctree(formula_regresie, constr, control = setari(md))
  data.frame(maxdepth = md, frunze = width(arbore),
             R2_constr = metrici_regresie(constr$tcp.len, predict(arbore))[[3]],
             t(metrici_regresie(test$tcp.len, predict(arbore, test))))
})), "adancime_regresie.csv")


# ---- 10.4 Arborele de clasificare (maxdepth 5) -----------------------------

m <- ctree(formula_clasificare, constr, control = setari(5))
cat("\n10.4 arborele de clasificare\n")
print(m)
cat("aceleasi predictii la maxdepth 6:",
    identical(predict(m, test), predict(ctree(formula_clasificare, constr, control = setari(6)), test)),
"\n")


# ---- 10.1 Testele din rădăcină ---------------------------------------------

# Statistica fiecărei variabile în rădăcină, cu p recalculat din chi-pătrat;
# log10p7 = p înmulțit cu 7 (corecția Bonferroni pentru cele 7 variabile).
cat("\n10.1 radacina: statistica, df, log10 p, log10 p*7\n")
statistici <- info_node(node_party(m))$criterion["statistic", ]
grade_libertate <- sapply(X7, function(v) if (is.factor(constr[[v]])) nlevels(droplevels(constr[[v]])) - 1 e
lse 1)
log10_p <- pchisq(statistici, grade_libertate, lower.tail = FALSE, log.p = TRUE) / log(10)
print(round(cbind(T = statistici, df = grade_libertate, log10p = log10_p, log10p7 = log10_p + log10(7))[orde
r(log10_p), ], 2))

```

```r
# Verificare de mână pentru mqtt.qos: statistica este chi-pătrat pe tabelul 2x2
# (qos > 0 față de atac), adică (n - 1) * r².
q <- table(constr$mqtt.qos > 0, constr$is_attack)
E <- outer(rowSums(q), colSums(q)) / sum(q)
cat("tabel qos: O\n")
print(q)
cat("E\n")
print(round(E, 1))
cat("chi2", sum((q - E)^2 / E),
    " (n-1) r2", (nrow(constr) - 1) * cor(constr$mqtt.qos, as.numeric(constr$is_attack))^2,
    " r", cor(constr$mqtt.qos, as.numeric(constr$is_attack)), "\n")

# Același arbore fără corecția Bonferroni
fara_bonferroni <- ctree(formula_clasificare, constr, control = setari(5, "Univariate"))
cat("\nfara Bonferroni: frunze", width(fara_bonferroni),
    " aceleasi predictii pe test:", identical(predict(fara_bonferroni, test), predict(m, test)), "\n")


# ---- 10.4 Frunzele arborelui de clasificare ---------------------------------

cat("\nfrunze: id | regula | n | % atac | prezice | clase\n")
frunza_constr <- predict(m, type = "node")
reguli <- partykit:::.list.rules.party(m)
for (id in sort(unique(frunza_constr))) {
  in_frunza <- frunza_constr == id
  clase <- sort(table(constr$target[in_frunza]), decreasing = TRUE)
  clase <- clase[clase > 0]
  cat(id, "|", reguli[as.character(id)], "|", sum(in_frunza), "|",
      round(100 * mean(constr$is_attack[in_frunza] == "atac"), 2), "|",
      as.character(predict(m, constr[which(in_frunza)[1], ])), "|",
      paste(names(clase), clase, collapse = "; "), "\n")
}


# ---- 10.5 Evaluarea pe test -------------------------------------------------

prezis <- predict(m, test)
cat("\n10.5 test\n")
print(table(real = test$is_attack, prezis = prezis))
cat("test", metrici_clasificare(test$is_attack, prezis),
    "\nconstructie", metrici_clasificare(constr$is_attack, predict(m)), "\n")
print(t(sapply(split(prezis != test$is_attack, test$target), function(x) c(n = length(x), gresite = sum
(x)))))
ratate <- test$is_attack == "atac" & prezis == "normal"
cat("atacuri ratate pe frunze:\n")
print(table(predict(m, test[ratate, ], type = "node")))


# ---- Verificări 10.2 / 10.4 -------------------------------------------------

cat("\nverificari 10.2 / 10.4\n")
# Toate variabilele care nu sunt constante (fără clasă, conținutul mesajului și id-ul lui)
X19 <- setdiff(names(d)[sapply(d, function(x) length(unique(x)) > 1)],
               c("target", "is_attack", "mqtt.msg", "mqtt.msgid"))
m19 <- ctree(reformulate(X19, "is_attack"), constr, control = setari(5))
cat(length(X19), "variabile: F1 test", metrici_clasificare(test$is_attack, predict(m19, test))[4],
    " radacina:", names(m19$data)[split_node(node_party(m19))$varid], "\n")

# tcp.len grupat în 3 intervale în loc de valoarea exactă
constr$len3 <- cut(constr$tcp.len, c(-1, 0, 20, Inf))
test$len3 <- cut(test$tcp.len, c(-1, 0, 20, Inf))
m_len3 <- ctree(reformulate(c(setdiff(X7, "tcp.len"), "len3"), "is_attack"), constr, control = setari(5))
cat("tcp.len in 0 / 1-20 / 21+: F1 test", metrici_clasificare(test$is_attack, predict(m_len3, test))[4],
"\n")


# ---- 10.6 Arborele de regresie pentru tcp.len (maxdepth 4) ------------------

r <- ctree(formula_regresie, constr, control = setari(4))
cat("\n10.6 arborele de regresie\n")
print(r)
prezis_reg <- predict(r, test)
eroare <- test$tcp.len - prezis_reg
cat("constructie", metrici_regresie(constr$tcp.len, predict(r)), "\ntest", metrici_regresie(test$tcp.len, pr
```

```r
ezis_reg), "\n")
cat("suma |e|", sum(abs(eroare)), " SSE", sum(eroare^2), " SST", sum((test$tcp.len - mean(test$tcp.len))^2),
"\n")
cat("cota SSE pe clase:\n")
print(round(tapply(eroare^2, test$target, sum) / sum(eroare^2), 4))

frunza_reg <- predict(r, type = "node")
cat("frunze regresie: id | n | media | mediana | clase\n")
for (id in sort(unique(frunza_reg))) {
  in_frunza <- frunza_reg == id
  clase <- table(droplevels(constr$target[in_frunza]))
  cat(id, "|", sum(in_frunza), "|", round(mean(constr$tcp.len[in_frunza]), 1), "|",
      median(constr$tcp.len[in_frunza]), "|", paste(names(clase), clase, collapse = "; "), "\n")
}

cat("\ncomparatie cu regresia liniara (R2 test: dreapta, arbore)\n")
for (f in list(mqtt.len ~ tcp.len, tcp.time_delta ~ tcp.len + mqtt.len)) {
  liniar <- lm(f, constr)
  arbore <- ctree(f, constr, control = setari(4))
  real <- test[[all.vars(f)[1]]]
  cat(deparse(f), metrici_regresie(real, predict(liniar, test))[3], metrici_regresie(real, predict(arbore, t
est))[3], "\n")
}


# ---- Tabelul nodurilor, pentru desene (10_grafice_arbori.py) ----------------

# Un rând pentru fiecare nod: id, copii (id-uri separate prin „;”), variabila
# despărțirii, p ajustat, etichetele ramurilor (separate prin „|”), n și
# conținutul nodului pe datele de construcție: numărul de pachete din fiecare
# clasă (clasificare) sau cuantilele și media lui tcp.len (regresie).
# Nodurile interne nu au rândurile lor în predict(type = "node"), așa că
# se adună frunzele de sub ele.
noduri <- function(arbore, y) {
  frunza <- predict(arbore, type = "node")
  do.call(rbind, lapply(nodeids(arbore), function(id) {
    nod <- nodeapply(arbore, id, identity)[[1]]  # id-urile din arborele întreg
    despartire <- split_node(nod)
    valori <- y[frunza %in% nodeids(arbore, id, terminal = TRUE)]
    rand <- data.frame(id = id, copii = "", var = "", p = NA, ramuri = "", n = length(valori))
    if (!is.null(despartire)) {
      rand$copii <- paste(sapply(kids_node(nod), id_node), collapse = ";")
      rand$var <- names(arbore$data)[despartire$varid]
      rand$p <- info_node(nod)$p.value
      rand$ramuri <- paste(character_split(despartire, arbore$data, digits = 6)$levels, collapse = "|")
    }
    continut <- if (is.factor(y)) t(as.matrix(table(valori))) else
      t(c(quantile(valori, c(0, .25, .5, .75, 1)), media = mean(valori)))
    cbind(rand, continut)
  }))
}
if (!is.null(folder_iesire)) {
  write.csv(noduri(m, constr$target), file.path(folder_iesire, "noduri_clasificare.csv"), row.names = FALSE)
  write.csv(noduri(r, constr$tcp.len), file.path(folder_iesire, "noduri_regresie.csv"), row.names = FALSE)
}
```

10_grafice_arbori.py

```
"""
Graficele din cap. 10 (arborii de decizie), desenate din CSV-urile scrise de
09_cap10_arbori_ctree.R.

Intrări (în folderul dat ca prim argument):
  noduri_clasificare.csv, noduri_regresie.csv  - tabelul nodurilor celor doi arbori
  adancime_clasificare.csv, adancime_regresie.csv  - metricile pe maxdepth 2-6 (10.3)

Ieșiri (în folderul dat ca al doilea argument):
  arbore_clasificare.png  - arborele is_attack; în fiecare frunză, bara cu clasele pachetelor
  arbore_regresie.png     - arborele tcp.len; în fiecare frunză, boxplot pe scară log
  adancime.png            - F1 / R² pe construcție și test, în funcție de maxdepth

Rulare:
  python 10_grafice_arbori.py folder_csv folder_iesire
"""
import re
import sys
import textwrap

import matplotlib
import numpy as np
import pandas as pd

matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.lines import Line2D
from matplotlib.patches import Patch, Rectangle

folder_csv = sys.argv[1]
folder_iesire = sys.argv[2]

CULORI_CLASE = {"legitimate": "#2a7ad6", "dos": "#e8692f", "bruteforce": "#1aae80",
                "malformed": "#f0a30a", "slowite": "#e377a6", "flood": "#008000"}


def format_zecimale(valoare, zecimale):
    """Format românesc: 12345.6 -> „12.345,6”."""
    text = f"{valoare:,.{zecimale}f}"
    return text.replace(",", " ").replace(".", ",").replace(" ", ".")


def format_scurt(valoare):
    return ("%g" % valoare).replace(".", ",")


def eticheta_ramura(text):
    """Eticheta unei ramuri din R: „<= 1e-06” -> „≤ 1e-06”; „0x00000018” -> „0x18”; listă -> „∈ {...}”."""
    text = re.sub(r"0x0+([0-9a-f]{2})", r"0x\1", text)
    prag = re.match(r"(<=|>) (.+)", text)
    if prag:
        semn = "≤ " if prag[1] == "<=" else "> "
        return semn + format_scurt(float(prag[2]))
    valori = text.split(", ")
    if len(valori) == 1:
        return "= " + valori[0]
    return textwrap.fill("∈ {" + ", ".join(valori) + "}", 16)


# ---------------------------------------------------------------------------
# Desenul unui arbore: nodurile interne sus, toate frunzele pe ultimul rând
# ---------------------------------------------------------------------------

def deseneaza_arbore(fisier_csv, titlu, fisier_png, deseneaza_frunza, legenda):
    noduri = pd.read_csv(fisier_csv, keep_default_na=False).set_index("id")
    copii = {i: [int(k) for k in str(nod.copii).split(";")] if nod.copii else [] for i, nod in noduri.iterro
ws()}

    # poziții: frunzele la x = 0, 1, 2, ... în ordinea parcurgerii,
    # fiecare nod intern deasupra mijlocului copiilor lui; y = -adâncimea
    x, y, frunze = {}, {}, []
```

```
 71
 72      def pozitioneaza(nod, adancime):
 73          y[nod] = -adancime
 74          if not copii[nod]:
 75              x[nod] = len(frunze)
 76              frunze.append(nod)
 77          else:
 78              for copil in copii[nod]:
 79                  pozitioneaza(copil, adancime + 1)
 80              x[nod] = np.mean([x[copil] for copil in copii[nod]])
 81
 82      pozitioneaza(1, 0)
 83      y_frunze = min(y.values()) - 0.6
 84
 85      fig, ax = plt.subplots(figsize=(17, 12))
 86      for nod in noduri.index:  # liniile și etichetele ramurilor, apoi caseta nodului
 87          if not copii[nod]:
 88              continue
 89          rand = noduri.loc[nod]
 90          for copil, eticheta in zip(copii[nod], rand.ramuri.split("|")):
 91              este_frunza = not copii[copil]
 92              y_copil = y_frunze if este_frunza else y[copil]
 93              y_capat = y_copil + (0.45 if este_frunza else 0)
 94              ax.plot([x[nod], x[copil]], [y[nod], y_capat], color="#999", lw=1, zorder=1)
 95              # eticheta mai aproape de copil, ca să nu se suprapună cu a fratelui
 96              f = 0.8 if este_frunza else 0.55
 97              ax.text(x[nod] + f * (x[copil] - x[nod]), y[nod] + f * (y_capat - y[nod]), eticheta_ramura(etich
eta),
 98                      ha="center", va="center", fontsize=9, bbox=dict(fc="white", ec="none", pad=0.5), zorder=
2)
 99          p = float(rand.p)
100          text_p = "p < 0,001" if p < 0.001 else "p = " + format_zecimale(p, 3)
101          ax.text(x[nod], y[nod], f"{nod}  {rand['var']}\n" + text_p, ha="center", va="center",
102                  fontsize=10, bbox=dict(boxstyle="round", fc="#eef3fb", ec="#7f9fd4"), zorder=3)
103
104      for nod in frunze:
105          ax.text(x[nod], y_frunze + 0.2, str(nod), ha="center", fontsize=10)
106          deseneaza_frunza(ax, x[nod], y_frunze, noduri.loc[nod])
107
108      ax.set_xlim(-0.8, len(frunze) - 0.2)
109      ax.set_ylim(y_frunze - 2.4, 0.5)
110      ax.axis("off")
111      ax.set_title(titlu, fontsize=13)
112      ax.legend(handles=legenda, loc="lower center", ncol=len(legenda), frameon=False,
113                bbox_to_anchor=(0.5, -0.02), fontsize=10)
114      fig.tight_layout()
115      fig.savefig(f"{folder_iesire}/{fisier_png}", dpi=100)
116      plt.close(fig)
117
118
119  def frunza_clasificare(ax, x, y_frunze, nod):
120      """Bara stivuită din frunză: din ce clase sunt pachetele."""
121      baza = y_frunze - 1.75
122      for clasa, culoare in CULORI_CLASE.items():
123          inaltime = 1.6 * nod[clasa] / nod.n
124          ax.add_patch(Rectangle((x - 0.25, baza), 0.5, inaltime, fc=culoare, ec="white", lw=0.8))
125          baza += inaltime
126      procent_atac = 100 * (1 - nod.legitimate / nod.n)
127      ax.text(x, y_frunze - 2.0, f"n = {format_zecimale(nod.n, 0)}\natac {format_zecimale(procent_atac, 1)}%",
128              ha="center", va="top", fontsize=9)
129
130
131  def inaltime_log(valoare):
132      """Scară logaritmică pentru tcp.len, de la 0 la 32768 (maximul tcp.len) pe 1,6 unități."""
133      return 1.6 * np.log10(1 + valoare) / np.log10(1 + 32768)
134
135
136  def frunza_regresie(ax, x, y_frunze, nod):
137      """Boxplot tcp.len în frunză: minim-maxim, Q1-Q3, mediana, media (valoarea prezisă)."""
138      baza = y_frunze - 1.75
139      ax.plot([x, x], [baza + inaltime_log(nod["0%"]), baza + inaltime_log(nod["100%"])], color="#444", lw=1)
140      # înălțime minimă, ca o cutie cu Q1 = Q3 să rămână vizibilă
141      inaltime_cutie = max(inaltime_log(nod["75%"]) - inaltime_log(nod["25%"]), 0.004)
142      ax.add_patch(Rectangle((x - 0.2, baza + inaltime_log(nod["25%"])), 0.4, inaltime_cutie, fc="#cfe2fb", ec
```

```
="#2a7ad6"))
    ax.plot([x - 0.2, x + 0.2], [baza + inaltime_log(nod["50%"])] * 2, color="#123f80", lw=2.5)
    ax.plot(x, baza + inaltime_log(nod.media), "D", color="#e8692f", ms=6)
    ax.text(x, y_frunze - 2.0, f"n = {format_zecimale(nod.n, 0)}\nmedia {format_zecimale(nod.media, 1)}",
            ha="center", va="top", fontsize=9)
    if x == 0:  # axa tcp.len, o singură dată, lângă prima frunză
        for valoare in [0, 10, 100, 1000, 10000]:
            ax.text(-0.55, baza + inaltime_log(valoare), format_zecimale(valoare, 0), ha="right", va="cente
r",
                    fontsize=8, color="#666")
            ax.plot([-0.52, -0.47], [baza + inaltime_log(valoare)] * 2, color="#666", lw=1)
        ax.plot([-0.5, -0.5], [baza, baza + 1.6], color="#666", lw=1)
        ax.text(-0.5, baza + 1.7, "tcp.len", ha="center", fontsize=8, color="#666")


# ---------------------------------------------------------------------------
# 10.4 și 10.6: cei doi arbori
# ---------------------------------------------------------------------------

deseneaza_arbore(f"{folder_csv}/noduri_clasificare.csv", "Arborele de clasificare (is_attack), maxdepth =
5",
                 "arbore_clasificare.png", frunza_clasificare,
                 [Patch(fc=culoare, label=clasa) for clasa, culoare in CULORI_CLASE.items()])
deseneaza_arbore(f"{folder_csv}/noduri_regresie.csv", "Arborele de regresie (tcp.len), maxdepth = 4",
                 "arbore_regresie.png", frunza_regresie,
                 [Patch(fc="#cfe2fb", ec="#2a7ad6", label="Q1 până la Q3"),
                  Line2D([], [], color="#123f80", lw=2.5, label="mediana"),
                  Line2D([], [], marker="D", color="#e8692f", ls="", label="media (valoarea prezisă)")])


# ---------------------------------------------------------------------------
# 10.3: alegerea adâncimii maxime
# ---------------------------------------------------------------------------

clasificare = pd.read_csv(f"{folder_csv}/adancime_clasificare.csv")
regresie = pd.read_csv(f"{folder_csv}/adancime_regresie.csv")
adancimi = clasificare.maxdepth
# (construcție, test, titlu, adâncimea aleasă); în CSV, f1.1 este F1 pe test
panouri = [(clasificare.f1, clasificare["f1.1"], "Arborele de clasificare: F1", 5),
           (regresie.R2_constr, regresie.R2, "Arborele de regresie: R²", 4)]

fig, axs = plt.subplots(1, 2, figsize=(15.3, 5.78))
for ax, (constructie, test, titlu, aleasa) in zip(axs, panouri):
    ax.plot(adancimi, constructie, "o-", color="#2a7ad6", lw=2.5, ms=8, label="construcție (70%)")
    ax.plot(adancimi, test, "o--", color="#e8692f", lw=2.5, ms=8, label="test (30%)")
    for adancime, valoare in zip(adancimi, test):
        ax.annotate(format_zecimale(valoare, 3), (adancime, valoare), textcoords="offset points", xytext=(0,
-16),
                    ha="center", fontsize=9)
    ax.axvline(aleasa, color="gray", ls=":")
    ax.text(aleasa + 0.08, ax.get_ylim()[0], f"ales: {aleasa}", color="gray", va="bottom")
    ax.set_title(titlu)
    ax.set_xlabel("maxdepth")
    ax.set_xticks(adancimi)
    ax.legend(frameon=False, loc="upper left")
    ax.yaxis.set_major_formatter(lambda v, _: format_scurt(round(v, 3)))
    ax.grid(axis="y", alpha=0.3)
    ax.spines[["top", "right"]].set_visible(False)
fig.tight_layout()
fig.savefig(f"{folder_iesire}/adancime.png", dpi=100)
plt.close(fig)
```